Cutting the Cord: Trust Without the TA

Four contributions for UAV-assisted infrastructure-less IoVs: TA-offline authentication, TUAV handover, quorum eviction and a tether-split key

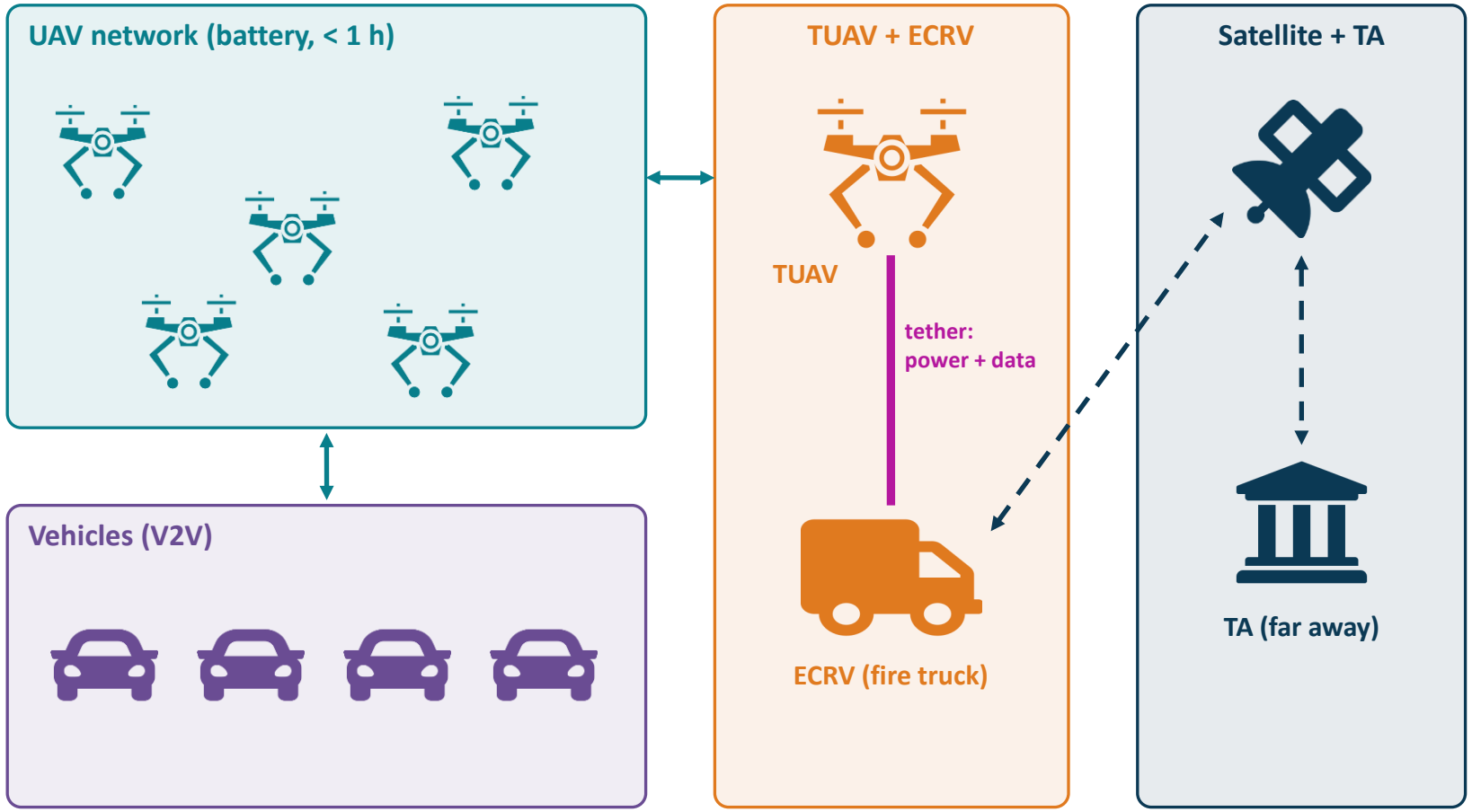
Phase 2 · Final presentation · Team 6

Swaraj Kumar	2025202016
Hritik Ranjan	2025202031
Harsh Raj	2025202005



Base paper: H. Tan, W. Zheng, P. Vijayakumar, “Secure and Efficient Authenticated Key Management Scheme for UAV-Assisted Infrastructure-Less IoVs,” IEEE Transactions on Intelligent Transportation Systems, vol. 24, no. 6, 2023

Phase 1 recap: a tethered drone replaces the broken roadside unit



Who does what

TA: issues each UAV's $\langle id_i, s_i \rangle$, keeps a copy, and checks every login with pairings.

TUAV: local base station; checks UAVs and hands out the group key.

UAVs: extend coverage; vehicles talk through them.

✗ roadside units (RSUs) down

How the base scheme works: five phases (Section IV)

IS1–IS5 · Setup	US1–US3 · UAV signs	GA1–GA4 · Authenticate	GD1–GD6 · Group key	DU1–DU2 · Update
TA	each UAV	TUAV ⇄ TA (satellite)	TUAV → UAVs	TUAV
TA → UAV i: $\langle id_i, s_i \rangle$ the TA keeps a copy	$\xi_i = h_1(r_i)$ (public) $\Gamma_i = [s_i h_3(\ldots) R_{tu} - h_1(s_i \xi_i)]P - \upsilon_i \xi_i Q_{tu}$	TA: 3 pairings per UAV TUAV: $\prod \psi_i (\prod \delta_i)^{h2} = \prod \mu_i^{Rtu}$	$\Lambda(x) = \kappa \cdot \Sigma \omega_i \varphi_i + \prod (x - \sigma_i)$ $\kappa = \Lambda(\sigma_i) \bmod \sigma_i$	join / leave → new $\Lambda^*(x)$ members keep their σ_i

The CRT trick (Chinese Remainder Theorem)

At a member's own key the product is 0, so $\Lambda(\sigma_i) = \kappa \cdot \Sigma \omega_i \phi_i$ — a number that leaves remainder κ modulo every σ_i.

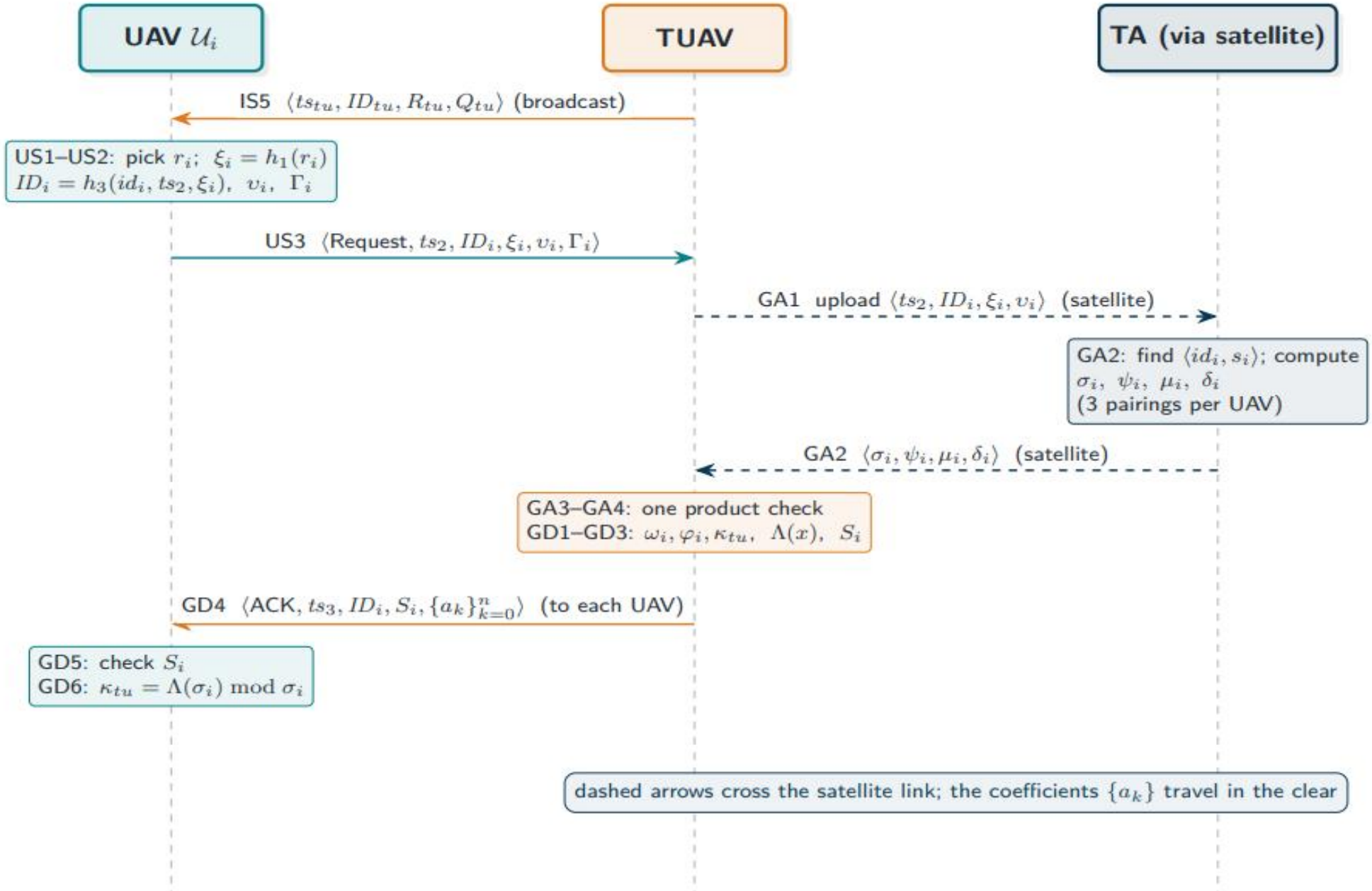
Example: σ = 7, 11, 13 and κ = 5 → $\Lambda(x) = x^3 - 31x^2 + 311x + 9014$.

$\Lambda(7) = \Lambda(11) = \Lambda(13) = 10015$, and 10015 mod 7 = mod 11 = mod 13 = 5 = κ.

What the paper claims (Sec. IV-D, Table II)

"…the malicious entities that successfully collect the coefficient set $\{a_i\}$ can not acquire the κ_{tu} without the correct decrypting key σ_i."

Table II: ✓ key-escrow resilience · ✓ forward secrecy · ✓ non-repudiation · ✓ multi-device revocation


UAV $\mathcal{U}_i$
TUAV
TA (via satellite)
IS5 $\langle ts_{tu}, ID_{tu}, R_{tu}, Q_{tu} \rangle$ (broadcast)
US1–US2: pick r_i; $\xi_i = h_1(r_i)$
$ID_i = h_3(id_i, ts_2, \xi_i)$, υ_i, Γ_i
US3 $\langle$Request, $ts_2, ID_i, \xi_i, \upsilon_i, \Gamma_i \rangle$
GA1 upload $\langle ts_2, ID_i, \xi_i, \upsilon_i \rangle$ (satellite)
GA2: find $\langle id_i, s_i \rangle$; compute
σ_i, ψ_i, μ_i, δ_i
(3 pairings per UAV)
GA2 $\langle \sigma_i, \psi_i, \mu_i, \delta_i \rangle$ (satellite)
GA3–GA4: one product check
GD1–GD3: $\omega_i, \varphi_i, \kappa_{tu}$, $\Lambda(x)$, S_i
GD4 $\langle$ACK, $ts_3, ID_i, S_i, \{a_k\}_{k=0}^{n} \rangle$ (to each UAV)
GD5: check S_i
GD6: $\kappa_{tu} = \Lambda(\sigma_i) \bmod \sigma_i$
dashed arrows cross the satellite link; the coefficients $\{a_k\}$ travel in the clear

Phase 2: three flaws we proved, four things we added

F1 · The TA can sign as any UAV

the TA keeps every s_i; the rest of Γ_i is public

F2 · The CRT key leaks — even to outsiders

one subtraction (member) or two broadcasts (anyone) reveal every σ_i

F3 · No forward secrecy

one leaked s_i opens every past group key

C1 · TA-offline authentication

certificateless keys, Schnorr-type signatures, batch check, one sealed key per member

→ removes F1, F2, F3

Once the TA leaves the field, the TUAV holds the trust → C2–C4 protect it

C2 · Handover between TUAVs

gap: one TUAV "covering the entire region" (IS4)

C3 · Voting out a compromised TUAV

gap: no way to revoke a TUAV with the TA offline

C4 · Tether-split key

gap: a captured TUAV keeps its whole key

A1–A7 · we attacked our own first design and fixed seven flaws — one of them (A7) was found only by formal verification

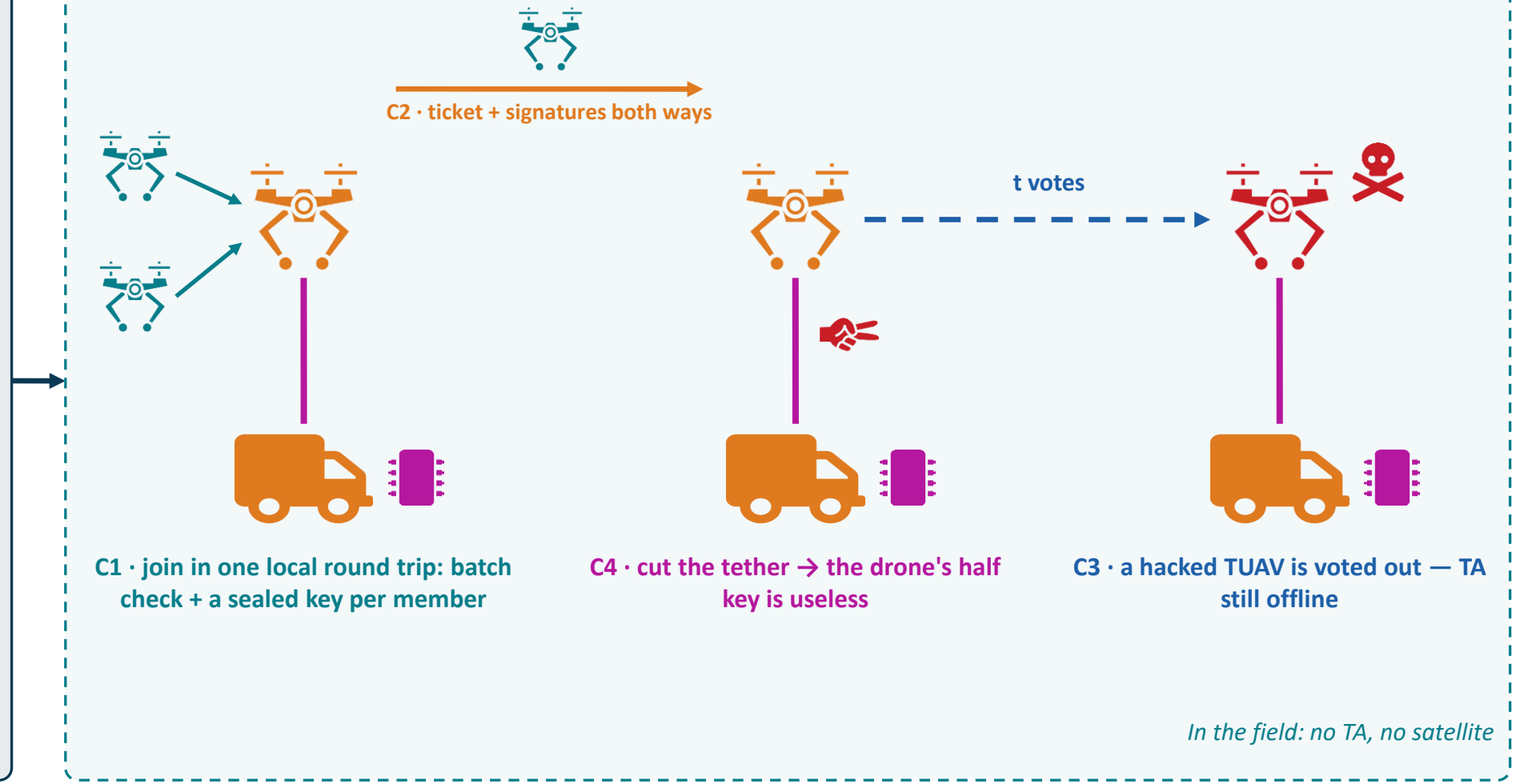
Our framework at a glance

Beforehand: setup by the TA

The TA issues certificateless credentials: sk = x + d.

x never leaves the device.

During the mission the TA is offline.



Flaw 1 · The TA can sign as any UAV (key escrow)

$$\Gamma_i = [\, s_i \cdot h_3(ID_i, ts_2, s_i) \cdot R_{tu} - h_1(s_i \xi_i) \,] \cdot P - \upsilon_i \cdot h_1(r_i) \cdot Q_{tu} \qquad (US2)$$

The only secret is s_i — and the TA issued it and stored a copy (IS2).

Input of Γ_i	UAV	TA	Public	Why
s_i	✓	✓	✗	issued + stored by TA
$\xi_i = h_1(r_i)$	✓	✓	✓	sent in clear (US3)
ID_i, υ_i, ts_2	✓	✓	✓	sent in clear (US3)
R_{tu}, Q_{tu}	✓	✓	✓	TUAV beacon (IS5)

Theorem 4.1 (TA forgery)

Whoever knows $\langle id_i, s_i \rangle$ logs in as UAV i:

1. pick any ξ′ (no r′ needed) and a fresh ts′
2. $ID' = h_3(id_i, ts', \xi')$, $\upsilon' = h_4(\ldots)$
3. Γ′ = the same formula with ξ′
4. GA1–GA4 accept → it also gets κ (GD6)

Breaks Table II: key-escrow resilience ✗ · non-repudiation ✗ (Girault level 1). The paper's Theorem 4 calls r_i the UAV's "partial key" — but r_i appears only as $\xi_i = h_1(r_i)$, which is sent in the clear.

Flaw 2 · One subtraction reveals every member's key

GD2 — only the constant term is masked

$$\Lambda(x) = \kappa \cdot \Sigma \omega_i \varphi_i + \prod (x - \sigma_i)$$

Lemma 5.1: $\Lambda(\sigma_j) = C$ for every member j (the product is 0 there)

$$\Lambda(x) - C = \prod (x - \sigma_i)$$

Theorem 5.2: integer roots (Rational Root Theorem, LLL) give every σ_i — a routine computation, not brute force.

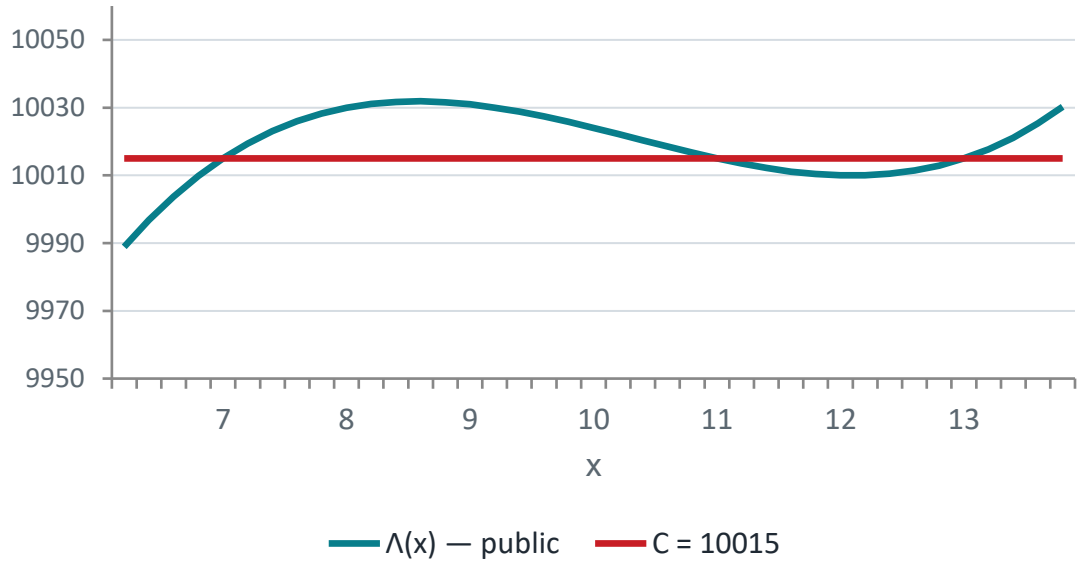
(a) Revocation fails

removed UAV:
$\kappa^* = \Lambda^*(\sigma_k) \bmod \sigma_k$
with a stolen σ_k

(b) Backward secrecy fails

new UAV:
old $\kappa = \Lambda(\sigma_j) \bmod \sigma_j$
from a recorded broadcast

Example: σ = 7, 11, 13 and κ = 5



$\Lambda(x) = x^3 - 31x^2 + 311x + 9014$ meets C = 10015 at 7, 11, 13.
Insider (σ = 7): $\Lambda(x) - 10015 = (x - 7)(x - 11)(x - 13)$.
After it is removed: $\Lambda^*(x) = x^2 - 24x + 1439 \rightarrow \Lambda^*(11) \bmod 11 = 9$ = the new key ✗

Flaw 2 · Even a passive outsider gets every key

Vieta's formulas (Lemma 5.5)

$$a_{n-1} = -(\sigma_1 + \sigma_2 + \ldots + \sigma_n) \quad \text{— not masked!}$$

Theorem 5.6 (passive outsider key recovery)

1. Record two broadcasts: before and after one UAV leaves (or joins).
2. Compare the sums: $\sigma = S - S'$ is the leaver's key.
3. Remove the mask: $q_0 = a_1 + \sigma \cdot a'_1$ (leave), $p_0 = a'_1 + \sigma \cdot a_1$ (join).
4. Roots → every σ_i → the old and the new group key.

Also checked at the paper's size: 160-bit keys, 12 UAVs, join and leave. Works for several leaves or joins at once (Prop. 5.7). Open: a single broadcast; mixed updates.

Example: keys 5, 7, 11 — the UAV with key 5 leaves

before: $\Lambda(x) = x^3 - 23x^2 + 167x + 1928$
after: $\Lambda'(x) = x^2 - 18x + 389$

① $\sigma = 23 - 18 = 5$
② $q_0 = 167 + 5 \cdot (-18) = 77$
③ $x^2 - 18x + 77 = (x - 7)(x - 11) \rightarrow$ keys $7, 11$
④ new key: $389 - 77 = 312, 312 \bmod 7 = 4$ ✓
old key: $2313 \bmod 7 = 3$ ✓

Paper, Sec. IV-D: "…the malicious entities that successfully collect the coefficient set $\{a_i\}$ can not acquire the κ_{tu} without the correct decrypting key σ_i." → contradicted word for word.

Flaw 3 · One leaked secret opens every past session

$$\sigma_i = h_1(s_i \cdot \xi_i) \cdot P$$

s_i: long-term secret · ξ_i: sent in the clear · nothing fresh and secret

session 1

recorded: $\xi^{(1)}$, $\{a_k^{(1)}\}$

session 2

recorded: $\xi^{(2)}$, $\{a_k^{(2)}\}$

session 3

recorded: $\xi^{(3)}$, $\{a_k^{(3)}\}$

later: s_i leaks

captured UAV or breached TA



$\kappa^{(1)}$ recovered ✗ $\kappa^{(2)}$ recovered ✗ $\kappa^{(3)}$ recovered ✗

$$\kappa^{(t)} = \Lambda^{(t)}(\sigma_i^{(t)}) \bmod \sigma_i^{(t)} \text{ for every past session } t$$

Theorem 6.1 (retroactive key recovery)

one leak of s_i → every past group key of that UAV; with Flaw 2, every other member's σ too

Table II claims forward secrecy ✓ → ✗

and the TA, which stores every s_i, can open the past at will

Contribution 1 · Authentication with the TA offline

PART 2 · MATHEMATICS

① Keys: certificateless

TA, before the mission:
$Y = yP,\ h = H_1(PID \| ep \| X \| Y)$
$d = y + s \cdot h$

UAV adds its own secret x:
$sk = x + d$
$pk = X + Y + h \cdot P_{pub}$

the TA never sees x

② Sign + batch check

$R = rP,\ c = H_2(m \| \ldots \| R)$
$v = r + c \cdot sk$
check: $vP = R + c \cdot pk$

many at once, random 64-bit λ_i:
$(\Sigma \lambda_i v_i)P = \Sigma \lambda_i R_i$
$+ \Sigma \lambda_i c_i (X_i + Y_i) + (\Sigma \lambda_i c_i h_i) P_{pub}$

batch fails → check one by one

③ One sealed key per member

$K_i = KDF(e_a E_i \| PID_i \| \ldots)$
$C_i = Enc(K_i, GK \| \nu)$

one broadcast: all C_i
fresh GK on every join and leave

e_a, e_i erased after use

F1 → the TA never sees x

F2 → no public polynomial

F3 → fresh e, erased

Contribution 1 · Security theorems and numbers

PART 2 · MATHEMATICS

Unforgeability (Thm 7.4)

Forking Lemma ⇒ ECDLP, for an outsider (Type I) and a malicious TA (Type II)

Batch soundness (Thm 7.5)

Bellare–Garay–Rabin test: a bad batch passes with probability ≤ 2^{-64}

No key escrow (Thm 7.6)

the TA cannot sign as a UAV: Girault level 2 (base paper: level 1)

Group-key secrecy (Thm 7.7)

outsider, removed member, newcomer learn nothing: CDH + AEAD

Forward secrecy (Thm 7.8)

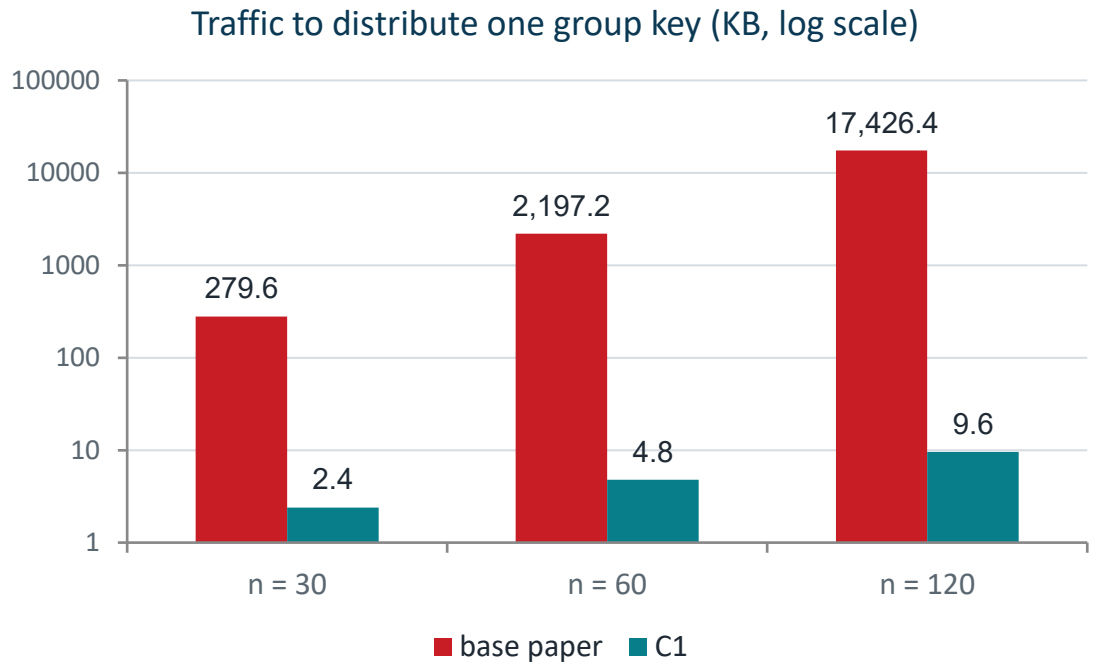
old keys need $e_a e_i P$: CDH on erased exponents

0

pairings per login round
(base paper: 3n at the TA)

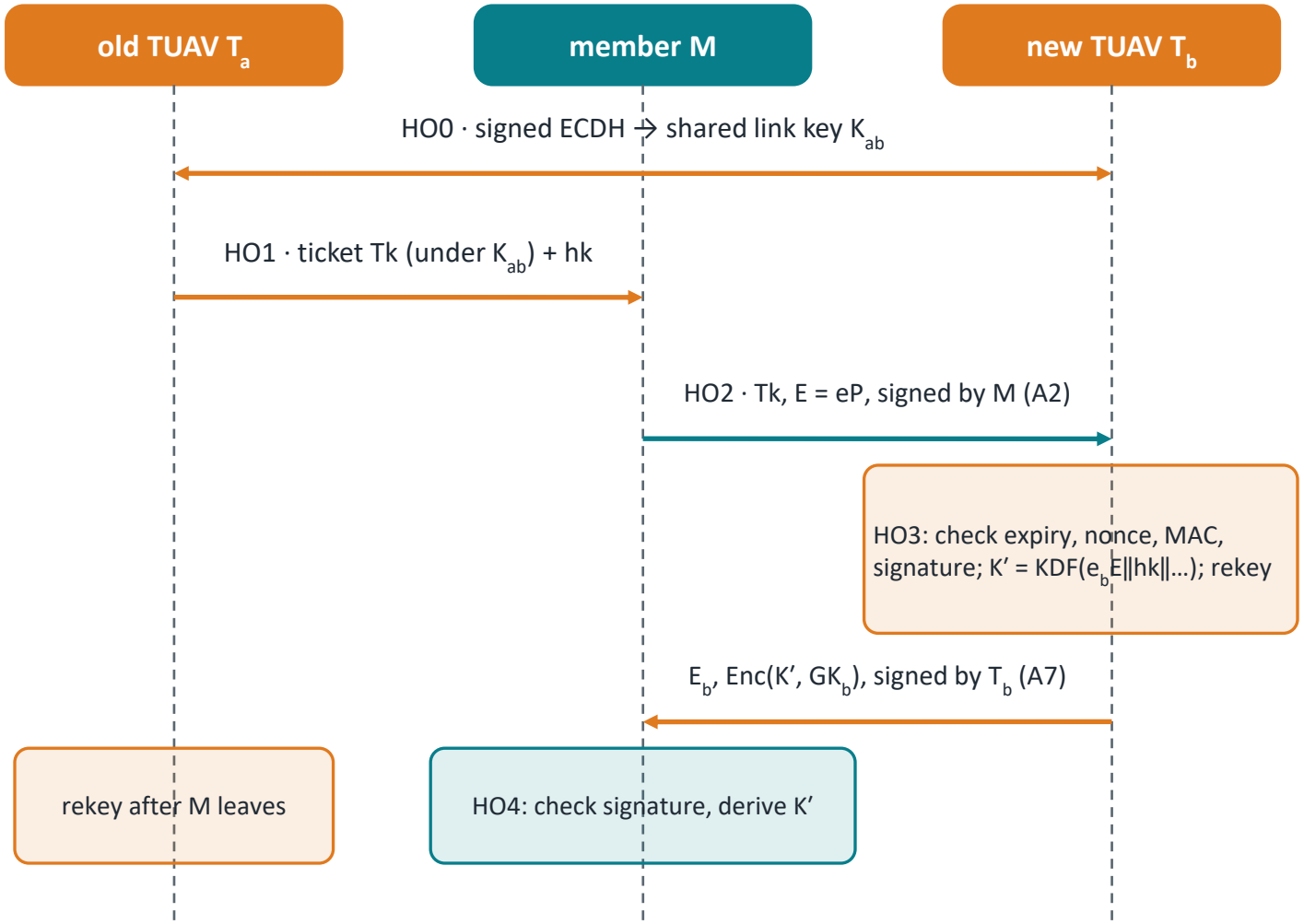
≈230 ms

vs ≈3720 ms + satellite trip
(120 UAVs, paper's own constants)



Contribution 2 · Handover between TUAVs

PART 3 · SECURITY



Gap in the paper

IS4: one TUAV "covering the entire region" — every move means a new login via the TA

Why it is secure

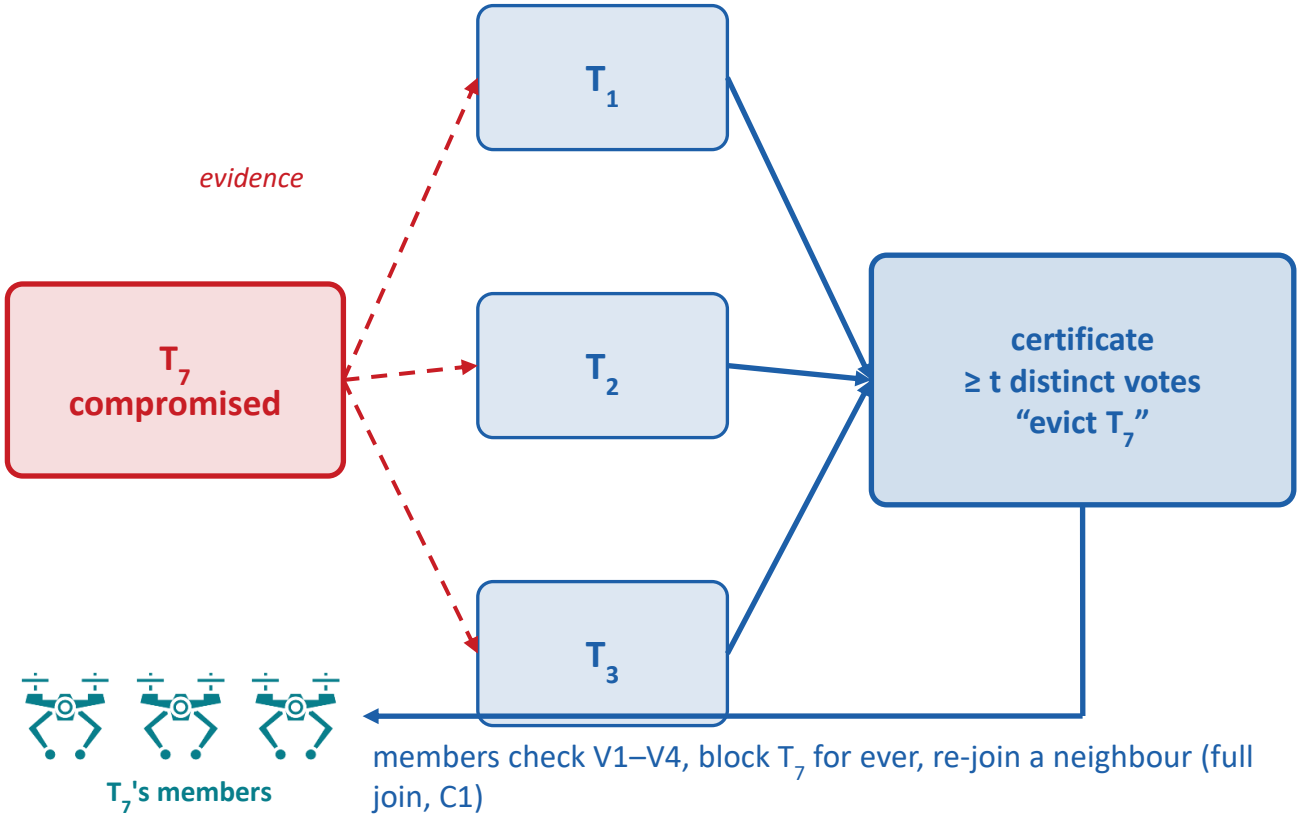
- hk, not K_M: the new TUAV never learns the old key
- fresh ECDH → forward secrecy
- nonce + expiry: a ticket works once
- signatures both ways (A2, A7)

Theorem 8.1 · formal verification

Dolev–Yao model, old TUAV captured: all 5 properties hold. No TA; saves the beacon wait (≈500 ms at a 1 s beacon).

Contribution 3 · Vote out a compromised TUAV

PART 3 · SECURITY



A valid certificate

V1 ≥ t distinct TUAV signers
V2 the target is not a signer
V3 every vote names the same target
V4 weighted batch check holds (TUAV role ep = 0)

Theorems 9.2 and 9.3

Safety: ≤ t − 1 compromised ⇒ an honest TUAV is never evicted.
Liveness: t honest voters ⇒ the certificate forms.
Choose t = f + 1; liveness needs $n_T \geq 2f + 1$.

Known pattern, new use

PBFT view change (Castro–Liskov 1999) — here for key management with the TA offline

Contribution 4 · Split the TUAV's key across the tether

PART 3 · SECURITY



drone: $a = x_A$

alone it signs nothing



cut → chip refuses + pre-signed kill notice



chip in ECRV:

$g = x_G + d$



pk = aP + gP — unchanged; nobody holds a + g

Joint Schnorr signature over the tether

$$sk = a + g$$
$$R = R_A + R_G, \quad c = H_2(m \parallel \ldots \parallel R)$$
$$v = (r_A + c \cdot a) + (r_G + c \cdot g)$$
$$vP = R + c \cdot pk \checkmark \quad \text{an ordinary signature: UAVs see no change}$$

Theorem 10.2 · half a key is useless

$(v - c \cdot a)P = R + c \cdot G$ with $G = gP \Rightarrow$ forging with $a =$ forging under $G \Rightarrow$ needs the ECDLP

Safe protocol

commit-then-reveal (no ROS attack) or FROST, RFC 9591

Cost

drone +0.012 ms; chip 0.90 ms in parallel; ≈200 bytes

Known vs new

mediated crypto (2001), FROST; new: helper = ECRV, switch = tether

We attacked our own design first: A1–A7

PART 3 · SECURITY

	Flaw in our first design	Fix in the final design	Found by
A1	one forged packet naming a UAV got that UAV evicted	a failed signature never evicts anybody	review
A2	a captured TUAV could mint handover tickets	the member signs its own request (HO2)	review + formal
A3	a captured TUAV could frame a member at a neighbour	evictions cross cells only with checkable evidence	review
A4	a credential for one role could claim another	the role is bound into the identity (ep inside h)	review
A5	a replayed old rekey could roll the key back	key versions may only increase	review
A6	a newcomer received the key already in use	rekey on every arrival	review
A7	a captured old TUAV could pose as the new TUAV	the new TUAV signs its reply (HO3)	**formal verification**

A7 survived three careful readings and was found only by the symbolic model — that is why we report formal verification.

Results · base paper vs our design

PART 3 · SECURITY

Property	Base paper	Our design
The TA cannot sign as a UAV	✗ level 1	✓ level 2
A removed UAV cannot read new keys	✗	✓
A new UAV cannot read old keys	✗	✓
An outsider learns nothing from broadcasts	✗	✓
Forward secrecy	✗	✓
Login works with the TA offline	✗	✓
Handover between TUAVs	—	✓
Remove a compromised TUAV	—	✓
A captured TUAV cannot impersonate the TUAV	✗	✓

6 of 6

attacks that apply to the base paper succeed — every one fails on ours

0 vs 3n

pairings per login round

≈1800×

less key traffic (9.6 KB vs 17.4 MB, 120 UAVs)

Simulation of the base scheme — screenshots

real BN128 pairings · paper's steps IS/US/GA/GD/DU · Team 6

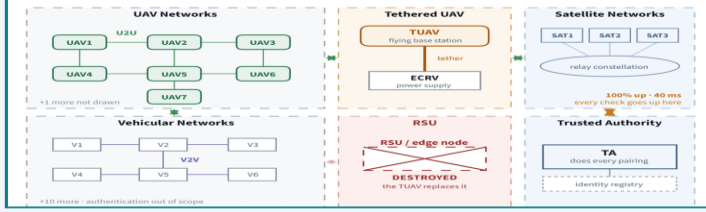
1 · System model running live

RSU destroyed, TUAV online, 8 UAVs registered; identities stay hidden

RSU **DESTROYED** — the premise
TUAV **ONLINE** — flying base station
TA LINK **100% · 40 ms** — on the critical path
UAVS **8** — registered
AUTHENTICATED **8** — of 8
REVOKED **0** — stale keys
GROUP KEY **EPOCH 1** — 8 holders
BACKEND **FAITHFUL** — BN128 (py_ecc)

Network | Protocol run | Group key | Attacks | Anonymity | Secure exchange | Security board | Performance | Paper vs us | Results | About the research

Live state | Animated walkthrough



Registered UAVs

UAV	Permanent ID	Temporary ID	Status	Key epoch
UAV1	hidden (DEFB4023)	TEMP_10835B4	Group key h	1
UAV2	hidden (F541EA0E)	TEMP_2FF1E47	Group key h	1
UAV3	hidden (F04E74EB)	TEMP_8915A0A	Group key h	1
UAV4	hidden (CFB1C612)	TEMP_16BCEB	Group key h	1
UAV5	hidden (5AB3C302)	TEMP_20EB8EI	Group key h	1
UAV6	hidden (869933BE)	TEMP_51673B6	Group key h	1
UAV7	hidden (C9C0D203)	TEMP_14F39F6	Group key h	1
UAV8	hidden (40F01CD4)	TEMP_18FFD0	Group key h	1

Permanent identities and secrets are never rendered. The fingerprint is a one-way label so you can follow an entity across sessions.

Event log

Time	Step	Actor	Event

2 · CRT group key works as printed

Epoch 1: all 8 members recover the same group key from one broadcast

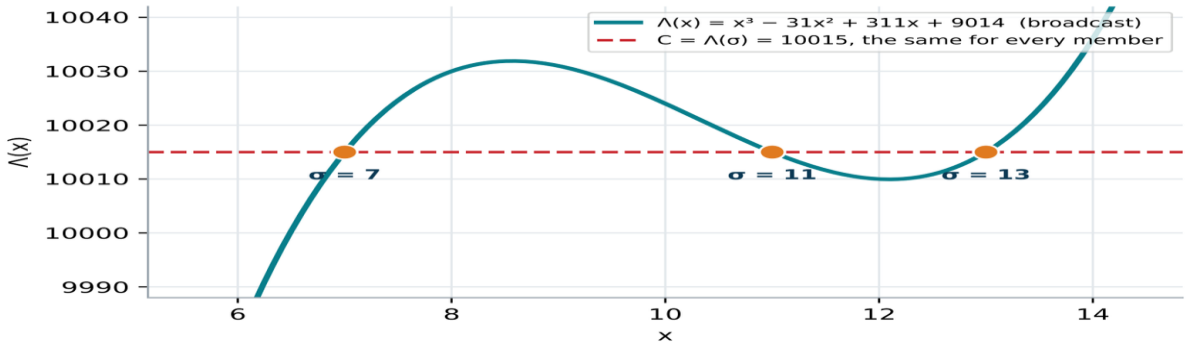
Epoch 1: 8 member(s) recovered the same group key from one broadcast.

Epoch	Members	Recovered key	Broadcast size
1	8	8	303 B

Declared correction. The paper defines sigma_i as `h1(s_i · x4_i)·P`, a point on the curve, then uses it as a modulus in Steps GD1 and GD6 — which is undefined for a curve point. We read sigma_i as an integer there and derive a prime from the same hash, which is the only reading under which the construction typechecks. Under that reading the paper's construction is correct and we implement it verbatim.

3 · Why the CRT key leaks (F2)

Λ(x) crosses the same value C at every member's key; subtract C → all keys



4 · TA link down → login stops

No local verification path in the base scheme: GA3 cannot run, nobody joins

RSU **DESTROYED** — the premise
TUAV **ONLINE** — flying base station
TA LINK **DOWN** — on the critical path
UAVS **8** — registered
AUTHENTICATED **0** — of 8
REVOKED **0** — stale keys
GROUP KEY **none** — not distributed

Group authentication failed. TA unreachable - Step GA3 cannot be evaluated. The scheme has no local verification path, so authentication does not degrade, it stops.

UAV signing (all n)	TUAV side	TA side	TA link
0.2 ms	0.0 ms	0.0 ms	40 ms

All 8 members were rejected and the culprit is unknown. Step GA4 of the paper reads only: "If matches, the validity of the vehicles can be proved." There is no else-branch anywhere in the paper.

5 · One bad request blocks everyone

Batch check fails, all 8 rejected, culprit unknown; the TA does 12× the TUAV's work

RSU **DESTROYED** — the premise
TUAV **ONLINE** — flying base station
TA LINK **100% · 40 ms** — on the critical path
UAVS **8** — registered
AUTHENTICATED **0** — of 8
REVOKED **0** — stale keys
GROUP KEY **none** — not distributed

Group authentication failed. aggregate equation did not balance - all 8 members rejected, culprit unknown. Step GA4 of the paper has no else-branch.

UAV signing (all n)	TUAV side	TA side	TA link
0.4 ms	0.3 ms	4.1 ms	40 ms

The pairings did not disappear — they moved. On this run the TA did **12×** the work the TUAV did. The paper reports only the TUAV side, which is what makes its group-authentication figure small.

All 8 members were rejected and the culprit is unknown. Step GA4 of the paper reads only: "If matches, the validity of the vehicles can be proved." There is no else-branch anywhere in the paper.

6 · What the simulator reproduces

Same steps and equations as the paper; σᵢ read as an integer (declared)

What this simulator is, and what it is not

Aspect	The paper	This simulator	Fidelity
Protocol flow	Steps IS/US/GA/GD/DU	Same steps, same labels	FAITHFUL
Bilinear pairing	supersingular curve, 80-bit	real BN128 via py_ecc, or a toy bilinear group	FAITHFUL / ABSTRACTED
Credential Gamma_i	as printed in Step US2	computed exactly as printed	FAITHFUL
Aggregate equation	Step GA3	same equation; correctness asserted by a unit test	FAITHFUL
CRT group key	sigma_i used as both point and modulus	sigma_i read as an integer; correction declared	INTERPRETED
Security theorems	formal proofs, Theorems 1-5	behavioural demonstrations only	NOT REPRODUCED
Benchmark environment	MIRACL (C), 80-bit	pure Python, BN128, this laptop	NOT COMPARABLE
TA-side cost	not reported	measured	NEW
TA link latency / availability	not modelled	modelled and swept	NEW
Vehicle tier	explicitly out of scope	drawn, not authenticated	OUT OF SCOPE

Conclusion

PART 3 · SECURITY

What we showed

- Three flaws, each a theorem with a worked example: TA impersonation, CRT key leakage (insider and passive outsider), no forward secrecy.
- A design that keys UAVs with the TA offline (C1), and protects the TUAV that inherits the trust: handover (C2), quorum eviction (C3), tether-split key (C4).
- Security backed by named results: Forking Lemma, Small-Exponent Test, CDH, PBFT-style quorum, formal verification.

Future work

- outsider attack from a single broadcast, or mixed joins and leaves
- a formal model of the tether-split signing protocol
- a test on a real tethered drone and ECRV

The paper cut the cable to the roadside unit but kept a far-away authority and a leaky polynomial inside every key. We take the authority out of the field, seal each member's key separately, and protect the drone that inherits the trust.

Thank you — questions?